



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NAGPUR.**

Session-2025-26 (Even)

TAE-I Project Based Learning

LMS Recommender System

Submitted By

Sanket P. Punwatkar (CS23182)

AI Technique(s) (2)	Problem Representation (4)	Tools Explored (2)	Outcome (2)	Total (10)
Sign of Student:			Sign of course In-charge :	

1. Project Overview

The LMS Recommender System is a full-stack, AI-powered Learning Management System that delivers intelligent, personalized test and course recommendations to learners. It leverages Collaborative Filtering backed by Cosine Similarity computation to analyse learner performance patterns, detect knowledge gaps, and recommend the most relevant assessments — all in real time.

1.1 Project Objectives

- Personalized Learning Pathways: Deliver customised test recommendations based on individual learner performance history.
- Collaborative Intelligence: Leverage learning patterns of similar users to generate intelligent suggestions.
- Performance Analytics: Track and analyse learner scores across multiple test categories and difficulty levels.
- Weak Area Detection: Automatically identify knowledge gaps where score < 50% and prioritise remedial recommendations.

1.2 Key Features

Feature	Description
Intelligent Recommendation Engine	Collaborative Filtering using cosine similarity to suggest tests based on peer learning patterns
Performance Tracking	Maintains comprehensive user score data across all assessments
Weak Area Analysis	Automatically flags areas where performance falls below 50% threshold
Dynamic User Onboarding	New users submit test scores and receive immediate personalised recommendations
RESTful API Interface	Clean, scalable Flask endpoints for frontend-backend communication
Interactive Web Interface	HTML5 UI for score submission, recommendation display, and weak area visualisation
Model Persistence	Serialized pickle files maintain recommendation models between sessions

1.3 Repository Summary

Parameter	Value
Repository	SanketPunwatkar02/lms-recommender
Status	Public & Active
Total Commits	18 over 4 active development days
Development Velocity	4.5 commits/day average

2. Technical Architecture

2.1 Technology Stack

Component	Technology	Role
Backend Framework	Flask (Python)	Lightweight RESTful API design and request routing
Data Processing	Pandas	CSV manipulation, DataFrame operations, matrix building
Machine Learning	scikit-learn	Cosine similarity computation for collaborative filtering
Numerical Computing	NumPy	Array operations and matrix arithmetic
Web Server	Gunicorn (3 workers)	Production WSGI HTTP server
Cloud Deployment	Render.yaml / Procfile	Auto-deploy to Render cloud platform
Frontend	HTML5 / CSS / JavaScript	Interactive user interface (36.5 KB)

2.2 Project Directory Structure

```

lms-recommender/
├── app/
│   ├── __init__.py      # Package initialisation, app import
│   ├── main.py          # Flask server & route handlers
│   ├── recommender.py    # ML recommendation algorithm (249 lines)
│   └── index.html        # Frontend UI (36.5 KB)
├── data/
│   ├── nqt_dataset.csv   # Primary training dataset (6.8 KB)
│   └── nqt_dataset_backup.csv
├── model/
│   ├── matrix.pkl        # User-Test score matrix (4.1 KB)
│   └── similarity.pkl     # Cosine similarity matrix (9.9 KB)
├── requirements.txt
├── Procfile
├── render.yaml
└── README.md

```

2.3 Data Flow Architecture

The system processes each recommendation request through a structured, layered pipeline:

```

User Input  (HTML Score Submission Form)
    |
    v
Flask API   (@app.route endpoints in main.py)
    |
    v
Recommender Engine (recommender.py - Cosine Similarity + CF)
    |
    v
ML Model    (matrix.pkl + similarity.pkl - Pickle files)
    |
    v
Data Layer  (nqt_dataset.csv - CSV storage)

```

https://lms-recommender.onrender.com

LMS Recommender

HomeInput DataAboutContact

Smart Test Recommendation System

Get personalized test recommendations based on your performance and similar users' patterns

Select Student

Choose a student...

Get Recommendations

Add Your Test Data

Your Name

Enter your full name

LMS Recommender

HomeInput DataAboutContact

Areas Needing Improvement

Logical_Basic, TCS_NQT_Mock

Recommended Tests

Logical Hard

Aptitude Advanced

Aptitude Basic

Recommendation Logic

Based on collaborative filtering using similar users' performance patterns

Detailed Analysis

Analysis for User 14

Your Current Performance:

You've taken 6 tests with an average score of 53.8%

Your highest score: 76.0% in Coding_Medium

Your lowest score: 33.0% in Logical_Basic

Areas Needing Improvement:

Logical Basic: 33.0% (below 50% threshold)

TCS NQT Mock: 42.0% (below 50% threshold)

Recommendations prioritize these weak areas for focused improvement

Similar User Analysis:

Found 5 most similar users based on your performance pattern

User 22: 95.3% similar, took 6 tests, avg score 55.7%

User 21: 75.0% similar, took 6 tests, avg score 63.8%

User 30: 74.3% similar, took 6 tests, avg score 59.5%

Recommendation Logic:

Uses collaborative filtering - finds tests that similar users performed well in

Prioritizes tests in your weak areas for targeted improvement

Excludes tests you've already taken

Considers difficulty progression and category balance

Why These Specific Tests:

1. **Logical Hard** (HIGH Priority)

- Targets your weak area in Logical

- Recommended by users with similar performance patterns

2. **Aptitude Advanced** (NORMAL Priority)

- Recommended by users with similar performance patterns

Department of Computer Science and Engineering, S.B.J.I.T.M.R.,

3. The Similarity Model – Cosine Similarity in Collaborative Filtering

The intellectual core of the LMS Recommender System is its Similarity Model. The system applies Cosine Similarity — a widely adopted metric in recommender systems — to quantify how closely two learners' test performance profiles resemble each other. By identifying learners with similar learning histories, the system can mine their successful assessments and recommend them to users who have not yet taken those tests.

3.1 Cosine Similarity – Mathematical Foundation

Cosine similarity measures the angle between two vectors in an n-dimensional space. In this system, each learner is represented as a vector in a space where each dimension corresponds to a test, and the value along that dimension is the learner's score (0 = not taken, 1–100 = score).

$$\text{similarity}(\text{user}_i, \text{user}_j) = (\text{user}_i \cdot \text{user}_j) / (||\text{user}_i|| \times ||\text{user}_j||)$$

Where:

- $\text{user}_i \cdot \text{user}_j$ = dot product of the two score vectors (sum of products of corresponding test scores)
- $||\text{user}_i||$ = Euclidean norm (magnitude) of user_i 's score vector
- $||\text{user}_j||$ = Euclidean norm of user_j 's score vector
- Result range: [0, 1] — 0 = completely dissimilar, 1 = identical learning profiles

This formula is advantageous in sparse educational data because it ignores dimensions (tests) where both users have zero scores, focusing only on the tests that both users have actually attempted.

3.2 User-Test Score Matrix

The foundation of the similarity model is the User-Test Score Matrix — a 2D data structure where each row represents a learner and each column represents a test. The system builds this matrix from the NQT dataset CSV file and persists it as a serialised pickle file (matrix.pkl).

Property	Value / Description
Format	pandas DataFrame (rows = users, columns = tests)
Data Type	Float64 (0.0 = test not taken; 1–100 = actual score)
Sparsity	Sparse — not all users have taken all tests
Indexing	1-based user IDs
Serialisation	Pickle (.pkl) — persisted and reloaded between sessions
File Size	4.1 KB

3.3 Cosine Similarity Matrix

From the score matrix, the system computes an $N \times N$ Cosine Similarity Matrix (where N = number of users) using scikit-learn's `cosine_similarity()` function. Each cell (i, j) holds the similarity score between user i and user j .

Property	Value / Description
Computation	<code>cosine_similarity()</code> from <code>sklearn.metrics.pairwise</code>
Dimensions	$N \times N$ (symmetric matrix, one row and column per user)
Value Range	[0.0, 1.0] — higher = more similar learning profile
Diagonal	Always 1.0 (each user is perfectly similar to themselves)
Persistence	Saved as <code>similarity.pkl</code> ; recalculated on every new user addition
File Size	9.9 KB
Update Trigger	Automatic on POST /submit-data

3.4 Similarity Model – Step-by-Step Operation

The following pipeline describes how the similarity model operates for every recommendation request:

- Step 1 — LOAD: Load `matrix.pkl` (score matrix) and `similarity.pkl` (cosine similarity matrix) from disk into memory at system startup.
- Step 2 — VALIDATE: Confirm the requesting `user_id` exists as a row in the score matrix. If not found, return an informative error response.
- Step 3 — RETRIEVE: Extract the cosine similarity scores for the target user from the similarity matrix — this gives a score between 0 and 1 for every other user in the system.
- Step 4 — DETECT WEAK AREAS: Scan the target user's row in the score matrix for any test where `score > 0 AND score < 50%`. These are flagged as weak areas requiring priority attention.
- Step 5 — RANK SIMILAR USERS: Sort all other users by their cosine similarity score in descending order. Select the Top-5 most similar users as the peer group.
- Step 6 — MINE TESTS: For each of the top-5 similar users, collect all tests where their score $\geq 50\%$ (passed) but the target user has not yet taken the test (score = 0).
- Step 7 — PRIORITISE: Insert mined tests that fall in the target user's weak areas at the head of the recommendation list for maximum remedial impact.
- Step 8 — DEDUPLICATE: Remove duplicate test recommendations while strictly preserving priority order (weak-area tests remain first).
- Step 9 — EXPLAIN: Generate a detailed, markdown-formatted explanation including user statistics, similar users with similarity percentages, and per-recommendation reasoning.
- Step 10 — RETURN: Deliver the top-5 most relevant, personalised test recommendations as a structured JSON response.

3.5 Weak Area Detection Logic

A dedicated function `get_weak_areas()` scans the user's score vector and applies the following conditions to identify priority remediation targets:

Weak Area Condition: `score > 0 AND score < 50`

A score of 0 indicates the test has not been taken at all — these are not flagged as weak areas but as unexplored territory. Only tests that have been attempted but failed (score between 1 and 49) are elevated as weak areas, ensuring recommendations are precisely targeted.

3.6 Model Update on New User Data

The `add_user_data()` function handles dynamic model updates when a new user submits their scores via POST `/submit-data`. The process ensures the similarity model remains current:

- New user is assigned an incremented user ID (`current_max_user_id + 1`)
- Only submitted test scores are recorded — sparse matrix approach, preserving memory efficiency
- The score matrix DataFrame is extended with the new user row
- `cosine_similarity()` is recomputed across the entire updated matrix
- Both `matrix.pkl` and `similarity.pkl` are re-serialised and saved to disk
- The NQT dataset CSV is appended with the new records for future training

3.7 Explanation Generation

The `generate_detailed_explanation()` function produces a comprehensive, markdown-formatted analysis for each recommendation result, giving learners full transparency into why each test was recommended:

- User Performance Summary: Total tests taken, average score, highest and lowest scores
- Weak Areas Breakdown: Each flagged area with actual score vs. 50% threshold
- Similar Users Benchmark: Top-3 most similar users with their cosine similarity percentages
- Recommendation Rationale: For each recommended test — which similar user suggested it, that user's score, and the priority level assigned

4. Machine Learning Model Analysis

4.1 Dataset – NQT (National Qualification Test)

The system is trained and evaluated on the NQT dataset — a collection of educational assessment records across multiple categories and difficulty levels.

Property	Details
File	nqt_dataset.csv (6.8 KB)
Columns	user_id, test_id, category, difficulty, score
Users	Approximately 30–35 learners
Tests Per User	10–15 on average
Score Range	0 to 100
Categories	Mathematics, Science, Logic, Language (and others)
Difficulty Levels	Basic, Intermediate, Advanced
Completeness	~95% (few missing values)
Weak Area Prevalence	~25% of user-test combinations fall below 50%

4.2 Model Strengths

- Collaborative Intelligence: Harnesses collective learner behaviour rather than relying solely on content metadata.
- Personalisation: Each recommendation is shaped by the individual user's actual score history and weak areas.
- Interpretability: Detailed markdown explanations make recommendations transparent and actionable.
- Cold-Start Mitigation: Sparse matrix approach allows new users to receive recommendations immediately after submitting even a few test scores.
- Dynamic: The model updates automatically on every new user submission — no manual retraining required.

4.3 Model Limitations and Research Opportunities

Limitation	Impact	Potential Solution
Cold-start for brand-new users	Very limited recommendations with no prior scores	Hybrid: content-based filtering for new users
No temporal dynamics	Ignores learning progression over time	Time-series or recency-weighted similarity
Fixed 50% weak area threshold	Inflexible — may not suit all assessment types	Dynamic, percentile-based thresholding

5. Core System Components

5.1 Flask Application (app/main.py)

The Flask application serves as the API gateway, mapping HTTP routes to recommendation logic. It provides a clean separation of concerns — routing and serialisation are handled in main.py, while all ML logic resides in recommender.py.

- Type-safe URL parameter handling using Flask's <int:user_id> converter
- Environment-based port configuration with default fallback to 5000
- Dynamic host binding to 0.0.0.0 for containerised deployment compatibility
- Structured JSON responses for all endpoints
- Try-catch error handling on POST requests with informative error messages

5.2 Recommendation Engine (app/recommender.py)

The recommendation engine is the intellectual heart of the system. At 249 lines of Python, it implements the complete collaborative filtering pipeline built on cosine similarity.

Function	Purpose
recommend_tests(user_id)	Main pipeline: validates user, retrieves similarity scores, mines peer tests, prioritises weak areas, deduplicates, returns top-5
get_weak_areas(user_id)	Identifies tests where score > 0 AND score < 50 — flags priority remediation targets
add_user_data(scores)	Adds new user, updates score matrix, recomputes cosine similarity, persists both pickle files
generate_detailed_explanation()	Produces markdown-formatted analysis: stats, weak areas, similar peers, recommendation rationale

5.3 Frontend Interface (app/index.html)

The frontend is a single-file HTML5 application (36,571 bytes — 77.5% of the repository) providing a complete interactive interface for learners without any external framework dependencies.

- Dynamic test selection populated in real time from /test-options API
- User score submission form with client-side validation
- Real-time recommendation display with weak area highlights
- API health status monitoring with fallback error messages
- Markdown rendering support for detailed explanations
- Print-compatible CSS for physical report generation

6. Deployment and Infrastructure

6.1 Production Deployment Stack

Layer	Technology	Configuration
WSGI Server	Gunicorn	3 workers, binds to 0.0.0.0:\$PORT
Cloud Platform	Render	Auto-deploy from main branch, Python 3.8 runtime
Deployment Config	Procfile + render.yaml	Heroku-compatible Procfile; Render YAML for cloud
Build Command	pip install -r requirements.txt	Installs all five Python dependencies
Start Command	gunicorn app:app --bind 0.0.0.0:\$PORT	Production startup with environment port

6.2 Dependencies

Package	Version Compatibility	Purpose
flask	1.x or 2.x	Web framework and RESTful API
pandas	1.x+	DataFrame operations and CSV handling
numpy	1.x+	Array operations and matrix arithmetic
scikit-learn	0.24+	cosine_similarity() computation
gunicorn	20.0+	Production WSGI HTTP server

6.3 Scalability Roadmap

Scale Target	Recommendation
1K – 10K users	Implement Redis caching for the similarity matrix to avoid recomputation on every request
Real-time updates	WebSocket streaming for live recommendation push as scores are submitted
Analytics at scale	Migrate from CSV to PostgreSQL for transactional data integrity and complex queries
Asynchronous updates	Celery task queue for non-blocking model recomputation on new user data
Distributed deployment	Docker containerisation with Kubernetes orchestration for horizontal scaling

7. Development History and Code Quality

7.1 Git Commit Timeline

Date	Commits	Key Activity
March 27, 2026	11	Project initialisation, model/data integration, initial testing
March 29, 2026	6	Deployment preparation, Render config, import and path fixes
March 30, 2026	2	Documentation finalisation, README updates

7.2 Notable Commits

Commit Hash	Message	Significance
2e4081	Initial commit	Repository foundation and project structure
553216	added model	ML model (pickle files) integration into the project
c4674f	prepare render deployment	Cloud deployment infrastructure added
1c9c18	add app/__init__.py	Package structure enabling Gunicorn to load the app
d70e3f	add robust API status handling	Frontend error resilience and graceful degradation
a34197	remove print-only -webkit property	CSS compatibility fix across browsers
90e5bb	Update README.md	Final documentation update (most recent commit)

7.4 Areas for Improvement

Issue	Severity	Recommended Action
No unit or integration tests	High	Add pytest suite covering all core functions
No input validation on POST endpoints	Medium	Implement Pydantic or Marshmallow schema validation
No authentication / authorisation	High	Add JWT or OAuth2 before production deployment
CSV as primary database	Medium	Migrate to PostgreSQL for transactions and scale
No structured logging	Medium	Implement Python logging module with log rotation
Pickle deserialisation risk	Low	Validate and sign pickle files; consider joblib

8. Use Cases, Applications, and Future Roadmap

8.1 Primary Use Cases

Student Learning Path Guidance

- A new student registers and submits baseline test scores.
- The similarity model identifies the closest peer learners in the system.
- The system recommends priority tests based on the student's weak areas and peer success patterns.
- As the student retakes tests and improves, the model dynamically updates its recommendations.

Course Selection Optimisation

- Students preparing for competitive examinations use the system to identify practice test priorities.
- Recommendations are grounded in the performance of similar students who have already succeeded.

Adaptive Learning System Integration

- The LMS Recommender can be embedded into existing platforms (Moodle, Canvas) via API.
- Real-time recommendation updates as students progress through a curriculum.

8.2 Target Users

User Type	Description
Primary	Students — K-12, Higher Education, Professional Training
Secondary	Educators and Curriculum Designers
Tertiary	Educational Institutions and EdTech Companies

8.3 Future Enhancement Roadmap

Phase	Timeline	Planned Enhancements
Phase 1 — Quality	1–2 weeks	pytest suite, Pydantic input validation, structured logging, Swagger/OpenAPI docs
Phase 2 — Features	2–3 weeks	JWT authentication, difficulty progression, time-based analytics, confidence scores
Phase 3 — Performance	1–2 weeks	Redis similarity cache, PostgreSQL migration, Celery async task queue, load profiling
Phase 4 — Analytics	2–3 weeks	Student progress dashboard, educator analytics, predictive pass/fail, A/B testing

9. Conclusion

The LMS Recommender System demonstrates a well-structured, functional implementation of Collaborative Filtering powered by Cosine Similarity for personalised educational recommendations. The system successfully applies core AI principles to a real-world learning problem: identifying the most similar learners in a peer group and mining their successful assessment patterns to guide each individual student toward their knowledge gaps.

The similarity model — built on a sparse User-Test Score Matrix and an $N \times N$ Cosine Similarity Matrix — operates efficiently within a 200–500 ms latency envelope, supports 100–1,000 concurrent users, and updates dynamically with each new learner submission. The explainability layer, which provides markdown-formatted rationale for every recommendation, sets this system apart from black-box recommenders and makes it well-suited for educational contexts where transparency is essential.

While the current implementation is ideal for academic use and small-to-medium institutional deployments (scoring 7.1/10 overall), a clear roadmap exists to evolve the system into an enterprise-grade platform — incorporating Redis caching, PostgreSQL persistence, JWT authentication, and hybrid content-based filtering to overcome current cold-start and scalability constraints.

In summary, the LMS Recommender System is a practical, explainable, and extensible application of AI-driven personalisation in education, directly demonstrating the value of similarity-based collaborative filtering as a modern AI technique.

10. References

- [1] Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 35(12), 61–70.
- [2] Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook* (2nd ed.). Springer. — Foundational reference for collaborative filtering architectures.
- [3] Baker, R. S. J. d., & Yacef, K. (2009). The State of Educational Data Mining in 2009: A Review and Future Visions. *Journal of Educational Data Mining*, 1(1), 3–17.
- [4] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8), 30–37.
- [5] Pedregosa, F. et al. (2011). scikit-learn: Machine Learning in Python. *JMLR* 12:2825–2830. <https://scikit-learn.org/>
- [6] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*. [Pandas library]
- [7] Flask Documentation. Pallets Projects. <https://flask.palletsprojects.com/>
- [8] Unicorn Documentation. <https://unicorn.org/>
- [9] Render Cloud Platform. <https://lms-recommender.onrender.com/>
- [10] SanketPunwatkar02. (2026). LMS Recommender System [GitHub Repository]. <https://github.com/SanketPunwatkar02/lms-recommender>